REVISION FLASHCARDS

DSA in Python

Dicts and sets to graphs and bit tricks

133 cards · 12 topics · question → answer

HOW TO USE THIS DECK

Every card is two slides.

1

Question — a dark slide. Answer out loud, or write the code on paper, *before* you flip.

2

Answer — the light slide that follows. Compare line by line, and note what you missed.

3

Navigate — every slide has an `index` link at the bottom right that jumps to the topic list.

Cards tagged **EXTRA** were added on top of your notes. Amber boxes (**WATCH OUT**) flag common slips.

TOPICS · CLICK TO JUMP

Index

01
Dicts & Sets
8 cards

02
Arrays
10 cards

03
Strings
13 cards

04
Sorting
16 cards

05
Linked Lists
7 cards

06
Queues & Deques
14 cards

07
Stacks
5 cards

08
Heaps & Priority Queues
10 cards

09
Binary Trees
14 cards

10
Binary Search Trees
8 cards

11
Graphs
18 cards

12
Bit Manipulation
10 cards

01

Dicts & Sets

Hash maps and hash sets: $O(1)$ on average, $O(n)$ in the worst case.

8 cards

QUESTION

What are the average vs. worst-case costs of dict access, insert, delete and `in`? What triggers the worst case?

Dict: core operations

Operation	Syntax	Average	Worst
Access	`d['a']` / `d.get('a')`	$O(1)$	$O(n)$
Insert / update	`d['c'] = 3`	$O(1)$	$O(n)$
Delete	`del d['a']` / `d.pop('a')`	$O(1)$	$O(n)$
Key exists	`'a' in d`	$O(1)$	$O(n)$
Get with default	`d.get('x', default)`	$O(1)$	$O(n)$
setdefault	`d.setdefault('x', [])`	$O(1)$	$O(n)$

- Worst case = heavy hash collisions. Rare in practice.

QUESTION

Which dict operations are $O(1)$ and which are $O(n)$: `keys()`, `len`, `popitem()`, `update`, `copy`, `clear`? What does `popitem()` remove?

Dict: the other operations

Operation	Cost
`d.keys()` / `values()` / `items()`	$O(1)$ to get the view, $O(n)$ to iterate
`len(d)`	$O(1)$
`d.popitem()`	$O(1)$
`d.update(other)`	$O(\text{len(other)})$
`d.copy()`	$O(n)$
`d.clear()`	$O(n)$ (every entry is released)

- `popitem()` removes the **last added** item (LIFO).

QUESTION

Count frequencies three ways. When is each one the right pick?

Frequency counting

- `Counter` is shortest when you only need counts.
- `defaultdict(int)`: the factory `int()` returns `0` for any missing key.
- `defaultdict(list)` is the go-to for grouping and adjacency lists.

```
# 1) vanilla dict
freq = {}
for x in arr:
    freq[x] = freq.get(x, 0) + 1

# 2) defaultdict
from collections import defaultdict
freq = defaultdict(int)
for x in arr:
    freq[x] += 1

# 3) Counter
from collections import Counter
freq = Counter(arr)
```

QUESTION

What does `most_common(2)` return for `Counter('aabbbc')`?

Counter & most_common

- `most_common(k)` returns the k highest counts as `(item, count)` pairs, highest first.

```
c = Counter("aabbbc")
print(c)
# Counter({'b': 3, 'a': 2, 'c': 1})
print(c.most_common(2))
# [('b', 3), ('a', 2)]
```

QUESTION

How do you create an empty set — and why can’t you write {}?

Creating sets

- `s = set()` is the empty set.
- `s = {1, 2, 3}` initialises with values.
- `{}` is an empty **dict**, not a set.

```
s = {1, 2, 3}   # with values
s = set()       # empty
```

QUESTION

Costs of single-element set operations: `add`, `remove`, `discard`, `in`, `pop`, `len`, `copy`?

Set: element operations

Operation	Syntax	Average	Worst
Add	`s.add(4)`	$O(1)$	$O(n)$
Remove (errors if missing)	`s.remove(4)`	$O(1)$	$O(n)$
Discard (no error)	`s.discard(4)`	$O(1)$	$O(n)$
Membership	`4 in s`	$O(1)$	$O(n)$
Pop arbitrary	`s.pop()`	$O(1)$	—
Length	`len(s)`	$O(1)$	—
Copy	`s.copy()`	$O(n)$	—

QUESTION

Write the operator for union, intersection, difference, symmetric difference and subset — and each one’s cost.

Set algebra

Operation	Syntax	Average cost
Union	`s1 | s2`	$O(\|s_1\| + \|s_2\|)$
Intersection	`s1 & s2`	$O(\min(\|s_1\|, \|s_2\|))$
Difference	`s1 - s2`	$O(\|s_1\|)$
Symmetric difference	`s1 ^ s2`	$O(\|s_1\| + \|s_2\|)$
Subset check	`s1 <= s2`	$O(\|s_1\|)$

QUESTION

`remove` vs `discard` vs `pop` — what happens when the element is missing, and which element does `pop` take?

remove / discard / pop

- `remove(x)` raises `KeyError` if `x` is missing.
- `discard(x)` does nothing if `x` is missing.
- `pop()` removes an **arbitrary** element — never rely on which one.

02

Arrays

Static vs dynamic arrays, list operation costs, and a dynamic array from scratch.

10 cards

QUESTION

Static vs dynamic arrays: what do they share and where do they differ?

Static vs dynamic arrays

- **Static:** fixed size chosen at creation, one contiguous block of memory.
- **Static:** index access $O(1)$; insert/delete in the middle $O(n)$ (shifting).
- `numpy.ndarray` is static: resizing means allocating a **new** array.
- **Dynamic:** same contiguous idea, but it can resize; Python lists are dynamic.
- **Dynamic:** when full, allocate a list of double the size and copy everything over.
- Insert at the front is $O(n)$ in **both**.

QUESTION

Why is a dynamic array's append $O(1)$ *amortized* rather than flat $O(1)$?

Amortized $O(1)$ append

- Most appends just write into spare capacity: $O(1)$.
- When full: allocate $2\times$ the space and copy all n items: $O(n)$.
- Doubling makes that expensive step rare, so the average cost per append stays $O(1)$.
- A static array has no append at all, so there is no amortization there.

QUESTION

What do `pop(0)` and `insert(0, x)` cost on a list, and what should you use instead?

Front operations on lists

- Both are $O(n)$: every element shifts by one.
- Use `collections.deque`: $O(1)$ at both ends.

QUESTION

Give the cost of `arr[i]`, `len`, a slice, `append`, `extend`, `insert`.

List costs: access & adding

Operation	Cost	Note
`arr[i]`	$O(1)$	direct index access
`len(arr)`	$O(1)$	the length is stored, not counted
`arr[a:b]`	$O(k)$	slicing creates a new list
`arr.append(x)`	$O(1)$ amortized	adds at the end
`arr.extend(it)`	$O(k)$ amortized	adds each element of `it`
`arr.insert(i, x)`	$O(n)$	everything from `i` onward shifts

QUESTION

Give the cost of `pop()`, `pop(i)`, `remove`, `del`, `clear`, `in`, `index`, `count`.

List costs: removing & searching

Operation	Cost	Note
`arr.pop()`	$O(1)$	from the end, no shift
`arr.pop(i)`	$O(n)$	everything past `i` shifts
`arr.remove(x)`	$O(n)$	find the first match, then shift
`del arr[i]`	$O(n)$	same cost as `pop(i)`
`arr.clear()`	$O(n)$	
`x in arr`	$O(n)$	linear search
`arr.index(x)`	$O(n)$	returns the position
`arr.count(x)`	$O(n)$	

QUESTION

Give the cost of `sort`, `sorted`, `reverse`, `arr[::-1]`, `arr1 + arr2`, `arr * k`.

List costs: reordering & combining

Operation	Cost	Note
`arr.sort()`	$O(n \log n)$	TimSort, stable, in place
`sorted(arr)`	$O(n \log n)$	returns a **new** list
`arr.reverse()`	$O(n)$	in place
`arr[::-1]`	$O(n)$	returns a new list
`arr1 + arr2`	$O(n + m)$	creates a new list
`arr * k`	$O(nk)$	

QUESTION

Does `arr[a:b]` return a view of the original list or a copy?

List slicing

- A **copy**: a brand-new list, costing $O(k)$ for a slice of length k.
- Changing the slice does not touch the original list.

QUESTION

Implement `DynamicArray` (1/3): the constructor, `get` and `set`.

DynamicArray: constructor, get, set

```
class DynamicArray:
    def __init__(self, capacity: int):
        self.capacity = capacity
        self.length = 0
        self.arr = [0] * self.capacity

    def get(self, i: int) -> int:
        if not 0 <= i <= self.length - 1:
            raise IndexError('Index out of range')
        return self.arr[i]

    def set(self, i: int, n: int) -> None:
        self.arr[i] = n
```

QUESTION

Implement `DynamicArray` (2/3): `pushback` and `popback`.

DynamicArray: pushback & popback

- Continues `class DynamicArray`.
- `pushback` grows the array first if `length == capacity`.

```
    def pushback(self, n: int) -> None:
        if self.length == self.capacity:
            self.resize()
        self.arr[self.length] = n
        self.length += 1

    def popback(self) -> int:
        if self.length == 0:
            raise IndexError('pop from empty array')
        self.length -= 1
        return self.arr[self.length]
```

QUESTION

Implement `DynamicArray` (3/3): `resize` and the size getters.

DynamicArray: resize & getters

- Continues `class DynamicArray`.
- `resize` doubles `capacity` and copies the `length` live items into the new array.

```
    def resize(self) -> None:
        self.capacity = 2 * self.capacity
        new_arr = [0] * self.capacity
        for i in range(self.length):
            new_arr[i] = self.arr[i]
        self.arr = new_arr

    def getSize(self) -> int:
        return self.length

    def getCapacity(self) -> int:
        return self.capacity
```

03

Strings

Immutability, slicing, the built-in methods worth knowing cold, and char/ASCII tricks.

13 cards

QUESTION

Strings are immutable. What happens on `s[0] = 'H'`, and what does that imply?

Immutability

- `TypeError`: strings don’t support item assignment.
- Every “modification” builds a **new** string.

```
s = "hello"
s[0] = 'H'
# TypeError: 'str' object does not
# support item assignment
```

QUESTION

Why is `s += c` in a loop $O(n^2)$, and what is the $O(n)$ pattern?

Building strings in a loop

- `+=` copies the whole string every time.
- Collect pieces in a list, then `join` once.

```
# bad: O(n^2)
result = ""
for c in some_list:
    result += c

# good: O(n)
result = []
for c in some_list:
    result.append(c)
result = "".join(result)
```

QUESTION

For `s = "hello world"`, evaluate `s[2:5]`, `s[::-1]`, `s[::2]`, `s[-1]`. What does slicing cost?

Access & slicing

```
s = "hello world"

s[0]       # 'h'            O(1)
s[-1]      # 'd'            O(1)
s[2:5]     # 'llo'          O(k), slicing copies
s[::-1]    # 'dlrow olleh'  O(n), reversed
s[::2]     # 'hlowrd'       every second char
len(s)     # 11             O(1)
```

QUESTION

Which methods change case? How do `title()` and `capitalize()` differ?

Case methods

```
s = "hello world"
s.upper()        # "HELLO WORLD"
s.lower()        # "hello world"
s.title()        # "Hello World"  every word
s.capitalize()   # "Hello world"  first letter only
s.swapcase()     # "HELLO WORLD"
```

QUESTION

Which methods strip whitespace — and what does `strip('xyz')` actually remove?

Cleaning whitespace

- `strip('xyz')` treats its argument as a **set of characters**, not as a substring.

```
s.strip()          # both ends
s.strip('xyz')     # any of x, y, z, both ends
s.lstrip()         # leading only
s.rstrip()         # trailing only
```

QUESTION

`find` vs `index` vs `in`: what happens when the substring is missing? What do `rfind`, `count`, `startswith` do?

Searching in strings

```
s.find('lo')          # first match; -1 if missing
s.rfind('lo')         # last match; -1 if missing
s.index('lo')         # like find, but ValueError
'lo' in s             # membership, O(n)
s.count('l')          # non-overlapping count, O(n)
s.startswith('hel')   # O(k)
s.endswith('rld')     # O(k)
```

QUESTION

How do you split on whitespace, on a delimiter, once only, from the right — and join back?

Splitting & joining

- `s.split()` also copes with repeated spaces and leading/trailing whitespace.

```
s = 'hello world'
s.split()                  # ['hello', 'world']
s.split(',')               # split on a delimiter
s.split(',', 1)            # maxsplit: only once
s.rsplit(',', 1)           # split from the right
s.splitlines()             # split on line breaks
','.join(['a', 'b', 'c'])  # 'a,b,c'  O(n)
```

QUESTION

Replace every occurrence of a character — and only the first one.

Replacing

```
s.replace('l', 'L')       # all occurrences, O(n)
s.replace('l', 'L', 1)    # only the first one
```

QUESTION

Name the six character-type checks.

Character-type checks

```
c.isalpha()    # letter
c.isdigit()    # digit
c.isalnum()    # letter or digit
c.isupper()    # uppercase
c.islower()    # lowercase
c.isspace()    # whitespace
```

QUESTION

Pad `'42'` with zeros to width 5. What do `ljust`, `rjust` and `center` do?

Padding & alignment

```
'42'.zfill(5)        # '00042'  zeros on the left
s.ljust(10, '*')     # left-justify, pad on the right
s.rjust(10, '*')     # right-justify, pad on the left
s.center(10, '*')    # center with padding
```

QUESTION

Convert between a character and its code point, and map `'a'`–`'z'` to indices 0–25.

ord / chr

```
ord('a')    # 97   character -> code point
chr(97)     # 'a'  code point -> character

# map 'a'..'z' <-> 0..25
index = ord(ch) - ord('a')
letter = chr(index + ord('a'))
```

QUESTION

How do you “mutate a string in place” in Python?

Strings and lists

- Convert to a list, edit, then `join` back.

```
s = 'hello'
chars = list(s)      # ['h','e','l','l','o']
chars[0] = 'H'
s = ''.join(chars)   # 'Hello'
```

QUESTION

How do strings compare, and why does `sorted("cab")` need a `join`?

Comparing & sorting strings

- `<` and `>` compare **lexicographically**, character by character.
- `sorted(s)` returns a **list** of characters, so `join` it back.

```
"abc" == "abc"            # True   value comparison
"abc" < "abd"             # True   lexicographic
sorted("cab")             # ['a', 'b', 'c']
''.join(sorted("cab"))    # 'abc'
```

04

Sorting

Complexity tables, stability, and every algorithm written out from memory.

16 cards

QUESTION

Bubble, selection, insertion sort: best / average / worst time, space, stable?

Simple $O(n^2)$ sorts

Algorithm	Best	Average	Worst	Space	Stable?
Bubble	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Selection	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No
Insertion	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes

QUESTION

Merge, quick, heap and Tim sort: best / average / worst time, space, stable?

$O(n \log n)$ sorts

Algorithm	Best	Average	Worst	Space	Stable?
Merge	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes
Quick	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No
Heap	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	No
Tim (Python's `sort()`)	$O(n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes

QUESTION

Counting, radix and bucket sort: time, space, stable? What is their catch?

Non-comparison sorts

Algorithm	Best	Average	Worst	Space	Stable?
Counting	$O(n+k)$	$O(n+k)$	$O(n+k)$	$O(n+k)$	Yes
Radix	$O(nk)$	$O(nk)$	$O(nk)$	$O(n+k)$	Yes
Bucket	$O(n+k)$	$O(n+k)$	$O(n^2)$	$O(n)$	Yes*

- Not comparison-based, so they can beat $O(n \log n)$ — but only for suitable keys (small integer ranges, digits, uniformly spread values).
- *Bucket sort is stable only if its inner sort is.

QUESTION

What does it mean for a sort to be *stable*? Which sorts are, and which aren’t?

Stability

- **Stable:** equal elements keep their original relative order.
- Stable: bubble, insertion, merge, counting, radix, bucket (if its inner sort is), Tim sort.
- **Not** stable: selection, quick, heap.

QUESTION

Which sorts reach $O(n)$ in the best case, and on what input?

Best-case $O(n)$

- **Bubble sort** — only with the `swapped` early exit, on already-sorted input.
- **Insertion sort** — on (nearly) sorted input.
- **Tim sort** — on data with existing sorted runs.

QUESTION

What does Python’s built-in `sort()` use, and what does it guarantee?

Python's `sort()` and `sorted()`

- **Tim sort:** $O(n \log n)$ average and worst case, $O(n)$ best case.
- Stable, with $O(n)$ extra space.
- `arr.sort()` sorts in place; `sorted(arr)` returns a new list.

QUESTION

Write bubble sort with the early-exit optimisation.

Bubble sort

- Repeatedly swap adjacent out-of-order pairs; each pass bubbles the largest unsorted element to its place.
- If a full pass makes no swap, the list is sorted — that flag gives the $O(n)$ best case.

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        swapped = False
        for j in range(n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True
        if not swapped:
            break           # already sorted
    return arr
```

QUESTION

Write selection sort.

Selection sort

- Find the minimum of the unsorted part and swap it into place.

```
def selection_sort(arr):
    n = len(arr)
    for i in range(n):
        min_idx = i
        for j in range(i + 1, n):
            if arr[j] < arr[min_idx]:
                min_idx = j
        arr[i], arr[min_idx] = arr[min_idx], arr[i]
    return arr
```

QUESTION

Write insertion sort.

Insertion sort

- Grow a sorted prefix one element at a time.
- Shift larger elements right, then drop `key` into the gap.

```
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1
        arr[j + 1] = key
    return arr
```

QUESTION

Merge sort (1/2): write the divide-and-conquer driver.

Merge sort: divide

- Split in half, sort each half recursively, merge.
- $O(n \log n)$ in every case, $O(n)$ extra space, stable.

```
def merge_sort(arr):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2

    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])

    return merge(left, right)
```

QUESTION

Merge sort (2/2): write `merge`. What do you do after the main loop?

Merge sort: merge

- Two pointers, always take the smaller front element.
- `<=` (not `<`) keeps the sort stable.

```
def merge(left, right):
    result = []
    i = j = 0

    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])

    return result
```

WATCH OUT After the loop only one side has leftovers. Extend with `left[i:]` and `right[j:]` — a classic slip is writing `left[j:]` twice.

QUESTION

Quick sort (1/2): write the recursive driver. What is the pivot idea?

Quick sort: driver

- Pick a pivot, put smaller elements to its left and the rest to its right, then recurse on both sides.
- Average $O(n \log n)$; worst case $O(n^2)$ (e.g. sorted input with a last-element pivot).

```
def quick_sort(arr, low=0, high=None):
    if high is None:
        high = len(arr) - 1
    if low < high:
        p = partition(arr, low, high)
        quick_sort(arr, low, p - 1)
        quick_sort(arr, p + 1, high)
    return arr
```

QUESTION

Quick sort (2/2): write `partition` with the last element as pivot.

Quick sort: partition

- `i` marks the end of the "≤ pivot" zone; the pivot finally swaps into `i + 1`, which is returned.

```
def partition(arr, low, high):
    pivot = arr[high]
    i = low - 1
    for j in range(low, high):
        if arr[j] <= pivot:
            i += 1
            arr[i], arr[j] = arr[j], arr[i]
    arr[i + 1], arr[high] = arr[high], arr[i + 1]
    return i + 1
```

QUESTION

Heap sort (1/2): write the sort itself. How do you build the max-heap, and what happens in the second loop?

Heap sort

- Build a max-heap by heapifying from `n // 2 - 1` down to `0`.
- Then repeatedly swap the max (root) to the end and shrink the heap.
- $O(n \log n)$, $O(1)$ space, not stable.

```
def heap_sort(arr):
    n = len(arr)
    for i in range(n // 2 - 1, -1, -1):
        heapify(arr, n, i)
    for i in range(n - 1, 0, -1):
        arr[0], arr[i] = arr[i], arr[0]
        heapify(arr, i, 0)
    return arr
```

QUESTION

Heap sort (2/2): write `heapify`. What are the child indices?

Heap sort: heapify

- Sift one node down inside a heap of size `n`.

```
def heapify(arr, n, i):
    largest = i
    left, right = 2 * i + 1, 2 * i + 2
    if left < n and arr[left] > arr[largest]:
        largest = left
    if right < n and arr[right] > arr[largest]:
        largest = right
    if largest != i:
        arr[i], arr[largest] = arr[largest], arr[i]
        heapify(arr, n, largest)
```

WATCH OUT Children of `i` are `2*i + 1` and `2*i + 2`. Writing a literal `2*2 + 2` for the right child is an easy bug.

QUESTION

Write counting sort. What input does it need?

Counting sort

- Count occurrences of each value, then rebuild the array in order.
- Needs non-negative integers with a small maximum k: $O(n + k)$ time and space.

```python
def counting_sort(arr):
    if not arr:
        return arr

    max_val = max(arr)
    count = [0] * (max_val + 1)

    for x in arr:
        count[x] += 1

    result = []
    for val, c in enumerate(count):
        result.extend([val] * c)
    return result
```

WATCH OUT This simple rebuild version works on plain integers. A truly stable version (for records) uses prefix sums.

05

Linked Lists

Node chains: pointer rewiring, head/tail trade-offs, and Floyd’s cycle check.

7 cards

QUESTION

Define `Node` and the start of `LinkedList`: constructor and `get(index)`. What does `get` return out of range?

Linked list: Node & get

- Out of range → returns `-1`.

```
from typing import List

class Node:
    def __init__(self, data=None):
        self.data = data
        self.next = None

class LinkedList:
    def __init__(self):
        self.head = None

    def get(self, index: int) -> int:
        temp = self.head
        i = 0
        while i < index and temp:
            temp = temp.next
            i += 1
        return temp.data if temp else -1
```

QUESTION

Write `insertHead` and `insertTail`. Which one is $O(1)$, and why?

Linked list: insertHead & insertTail

- Continues `class LinkedList`.
- `insertHead` is $O(1)$; `insertTail` is $O(n)$ because there is no tail pointer.

```
    def insertHead(self, val: int) -> None:
        temp = self.head
        self.head = Node(val)
        self.head.next = temp

    def insertTail(self, val: int) -> None:
        if not self.head:
            self.head = Node(val)
            return
        temp = self.head
        while temp.next:
            temp = temp.next
        temp.next = Node(val)
```

QUESTION

Write `getValues`: collect every value of the list into a Python list.

Linked list: getValues

- Continues `class LinkedList`.
- Walk from `head` until `None`: $O(n)$.

```
    def getValues(self) -> List[int]:
        result = []
        curr = self.head
        while curr:
            result.append(curr.data)
            curr = curr.next
        return result
```

QUESTION

Write `remove(index)` for a singly linked list. Which node do you stop at?

Linked list: remove

- Continues `class LinkedList`.
- Index 0 moves the head. Otherwise stop at the node **before** the target, then skip it: `curr.next = curr.next.next`.
- Return `False` when the index is out of range.

```
    def remove(self, index: int) -> bool:
        if not self.head or index < 0:
            return False
        if index == 0:
            self.head = self.head.next
            return True
        curr = self.head
        i = 0
        while curr.next and i < index - 1:
            curr = curr.next
            i += 1
        if curr.next is None:
            return False
        curr.next = curr.next.next
        return True
```

QUESTION

Time cost of `get`, `insertHead`, `insertTail`, `remove`, `getValues` on this list (head pointer only)?

Linked list: complexities

Operation	Cost	Why
`get(i)`	$O(n)$	walk `i` nodes
`insertHead`	$O(1)$	just rewire the head
`insertTail`	$O(n)$	walk to the last node
`remove(i)`	$O(n)$	walk to the node before `i`
`getValues`	$O(n)$	visit every node

QUESTION

Reverse a singly linked list. What do you do with the three pointers?

Reverse a linked list

- Save `next` **before** rewiring, point `current` back at `prev`, then advance both.
- `prev` ends up as the new head. $O(n)$ time, $O(1)$ space.

```
def reverseLL(head: Node) -> Node:
    prev, current = None, head

    while current:
        nextNode = current.next
        current.next = prev
        prev = current
        current = nextNode

    return prev
```

QUESTION

Detect a cycle in a linked list. How far does each pointer move per step, and why does it work?

Detect a cycle (Floyd)

- `slow` moves 1 step, `fast` moves 2.
- With a cycle, `fast` laps `slow` and they meet. Without one, `fast` reaches `None`.
- $O(n)$ time, $O(1)$ space.

```python
def detectCycle(head: Node) -> bool:
    slow = fast = head

    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next

        if slow == fast:
            return True

    return False
```

WATCH OUT `fast` must advance with `fast.next.next`. If both pointers move one step, they are always equal.

06

Queues & Deques

FIFO with `deque`, queues from linked lists and stacks, circular queues, double-ended queues.

14 cards

QUESTION

What is a queue, why is `list.pop(0)` a poor queue, and what do you use instead?

Queue basics

- **FIFO** — first in, first out. Heavily used in BFS.
- `list.pop(0)` is $O(n)$ (everything shifts), so plain lists don't scale.
- Use `collections.deque` (pronounced "deck").

QUESTION

Which deque operations are $O(1)$ and which are $O(n)$?

deque: costs

Operation	Cost	Note
`append(x)` / `appendleft(x)`	$O(1)$	add right / left
`pop()` / `popleft()`	$O(1)$	remove right / left
`extend(it)` / `extendleft(it)`	$O(k)$	`extendleft` reverses the order
`rotate(n)`	$O(k)$	negative `n` rotates left
`len(d)`	$O(1)$	
`d[i]`	$O(n)$	not built for random access
`insert` / `remove` / `index` / `count`	$O(n)$	
`reverse` / `clear` / `copy`	$O(n)$	

QUESTION

What does `deque(maxlen=3)` do when you append a 4th element? Where is it used?

Bounded deque

- Items fall off the **opposite** end once full.
- Heavily used in sliding-window problems.

```
from collections import deque

d = deque(maxlen=3)
d.append(1); d.append(2); d.append(3); d.append(4)

print(d)   # deque([2, 3, 4])  -- 1 dropped off
```

QUESTION

Implement a queue with a linked list (1/2): `Node`, constructor, `enqueue`.

Queue (linked list): enqueue

- Keep `front` and `rear` pointers so both ends are $O(1)$.

```
class Node:
    def __init__(self, val):
        self.val = val
        self.next = None

class Queue:
    def __init__(self):
        self.front = None
        self.rear = None
        self.size = 0

    def enqueue(self, val):
        newNode = Node(val)
        if not self.rear:
            self.front = self.rear = newNode
        else:
            self.rear.next = newNode
            self.rear = newNode
        self.size += 1
```

QUESTION

Implement a queue with a linked list (2/2): `dequeue`, `peek`, `is_empty`. What happens to `rear` when the queue empties?

Queue (linked list): dequeue

- Continues `class Queue`.
- When `front` becomes `None`, reset `rear` to `None` too.

```
    def dequeue(self) -> int:
        if not self.rear:
            raise IndexError("dequeue from an empty queue")
        val = self.front.val
        self.front = self.front.next
        if not self.front:
            self.rear = None
        self.size -= 1
        return val

    def peek(self) -> int:
        if not self.front:
            raise IndexError("peek from an empty queue")
        return self.front.val

    def is_empty(self) -> bool:
        return self.front is None
```

QUESTION

Build a queue from two stacks (1/2): the idea, enqueue and dequeue.

Queue from two stacks: dequeue

- Push into `in_stack`. To dequeue, if `out_stack` is empty, pour all of `in_stack` into it — the order reverses.
- Each element moves at most twice: amortized $O(1)$.

```
class QueueUsingStacks:
    def __init__(self):
        self.in_stack = []
        self.out_stack = []

    def enqueue(self, val) -> None:
        self.in_stack.append(val)

    def dequeue(self) -> int:
        if not self.out_stack:
            while self.in_stack:
                self.out_stack.append(self.in_stack.pop())
        if not self.out_stack:
            raise IndexError("dequeue from an empty queue")
        return self.out_stack.pop()
```

QUESTION

Build a queue from two stacks (2/2): `peek` and `is_empty`. What must `is_empty` check?

Queue from two stacks: peek & is_empty

- Continues `class QueueUsingStacks`.

```
    def peek(self) -> int:
        if not self.out_stack:
            while self.in_stack:
                self.out_stack.append(self.in_stack.pop())
        if not self.out_stack:
            raise IndexError("peek from an empty queue")
        return self.out_stack[-1]

    def is_empty(self) -> bool:
        return not self.in_stack and not self.out_stack
```

WATCH OUT `is_empty` must check **both** stacks. Checking only `in_stack` reports empty while items still sit in `out_stack`.

QUESTION

Circular queue: how do you compute the rear slot, and how do you move `front`?

Circular queue: the trick

- Fixed-size array of `capacity` slots, plus `front` and `count`.
- Rear slot: `(front + count) % capacity`.
- Dequeue: `front = (front + 1) % capacity`.
- Full when `count == capacity`; empty when `count == 0`.

QUESTION

Implement a circular queue (1/2): constructor and enqueue.

Circular queue: enqueue

```
class CircularQueue:
    def __init__(self, k):
        self.q = [None] * k
        self.capacity = k
        self.front = 0
        self.count = 0

    def enqueue(self, val) -> None:
        if self.count == self.capacity:
            raise OverflowError("queue is full")
        rear = (self.front + self.count) % self.capacity
        self.q[rear] = val
        self.count += 1
```

QUESTION

Implement a circular queue (2/2): `dequeue`, `is_empty`, `is_full`.

Circular queue: dequeue

- Continues `class CircularQueue`.

```
    def dequeue(self) -> int:
        if self.count == 0:
            raise IndexError("Empty Queue")
        val = self.q[self.front]
        self.front = (self.front + 1) % self.capacity
        self.count -= 1
        return val

    def is_empty(self) -> bool:
        return self.count == 0

    def is_full(self) -> bool:
        return self.count == self.capacity
```

QUESTION

What is a double-ended queue, and which Python class is one?

Deque basics

- Append **and** pop at both left and right ends in $O(1)$.
- Python's `collections.deque` is a double-ended queue.
- Built from a **doubly** linked list (`prev` and `next` pointers).

QUESTION

Implement a deque with a doubly linked list (1/3): `Node`, constructor, `append`.

Deque: constructor & append

```
class Node:
    def __init__(self, val):
        self.val = val
        self.next = None
        self.prev = None

class Deque:
    def __init__(self):
        self.front = None
        self.rear = None
        self.size = 0

    def append(self, val: int) -> None:
        newNode = Node(val)
        if not self.rear:
            self.front = self.rear = newNode
        else:
            newNode.prev = self.rear
            self.rear.next = newNode
            self.rear = newNode
        self.size += 1
```

QUESTION

Implement a deque (2/3): `appendleft` and `pop`.

Deque: appendleft & pop

- Continues `class Deque`.
- `pop`: move `rear` back; if the deque is now empty, clear `front` too, else cut `rear.next`.

```
    def appendleft(self, val: int) -> None:
        newNode = Node(val)
        if not self.rear:
            self.front = self.rear = newNode
        else:
            newNode.next = self.front
            self.front.prev = newNode
            self.front = newNode
        self.size += 1

    def pop(self) -> int:
        if not self.rear:
            raise IndexError("empty queue")
        val = self.rear.val
        self.rear = self.rear.prev
        if not self.rear:
            self.front = None
        else:
            self.rear.next = None
        self.size -= 1
        return val
```

QUESTION

Implement a deque (3/3): `popleft` and `is_empty`. Is every operation really $O(1)$?

Deque: popleft & is_empty

- Continues `class Deque`.
- Yes — every operation here is $O(1)$.

```
    def popleft(self) -> int:
        if not self.rear:
            raise IndexError("empty queue")
        val = self.front.val
        self.front = self.front.next
        if not self.front:
            self.rear = None
        else:
            self.front.prev = None
        self.size -= 1
        return val

    def is_empty(self) -> bool:
        return self.size == 0
```

07

Stacks

LIFO with lists, deque, and a linked-list implementation.

5 cards

QUESTION

What is a stack, and which built-in types can you use as one?

Stack basics

- **LIFO** — last in, first out.
- A Python `list` works as a stack: `append` and `pop` at the end.
- `collections.deque` also works.

QUESTION

Stack with a plain list: push, pop, peek, size, is-empty.

Stack with a list

```
stack = []
stack.append(1)
stack.append(2)
stack.append(3)

print(stack.pop())    # 3   pop the top
print(stack[-1])      # 2   peek
print(len(stack))     # 2   size
print(not stack)      # False   is_empty check
```

QUESTION

Stack with `collections.deque` — what do push / pop / peek look like?

Stack with deque

```
from collections import deque

stack = deque()
stack.append(1)
stack.append(2)
stack.append(3)

print(stack.pop())    # 3
print(stack[-1])      # 2   peek
```

QUESTION

Implement a stack with a linked list (1/2): `Node`, constructor, `push`.

Stack (linked list): push

- The `top` pointer is the head: push and pop are both $O(1)$.

```
class Node:
    def __init__(self, val):
        self.val = val
        self.next = None

class Stack:
    def __init__(self):
        self.top = None
        self.size = 0

    def push(self, val: int) -> None:
        newNode = Node(val)
        if not self.top:
            self.top = newNode
        else:
            newNode.next = self.top
            self.top = newNode
        self.size += 1
```

QUESTION

Implement a stack with a linked list (2/2): `pop`, `peek`, `is_empty`.

Stack (linked list): pop & peek

- Continues `class Stack`.

```
    def pop(self) -> int:
        if not self.top:
            raise IndexError("Empty Stack")
        val = self.top.val
        self.top = self.top.next
        self.size -= 1
        return val

    def peek(self) -> int:
        if not self.top:
            raise IndexError("Empty Stack")
        return self.top.val

    def is_empty(self) -> bool:
        return self.top is None
```

08

Heaps & Priority Queues

heapq, the max-heap trick, and a binary heap stored in an array.

10 cards

QUESTION

How does a priority queue differ from a stack or a queue?

Priority queue basics

- It ignores insertion order and serves by **priority** order.
- A binary heap implements it: $O(\log n)$ push and pop, $O(1)$ peek.

QUESTION

Push 5, 1, 3 into a heapq and pop twice. Where is the smallest element for peeking?

heapq basics

- `heapq` turns a plain Python `list` into a **min**-heap in place.
- Peek with `heap[0]`, **not** `heap[-1]`.

```
import heapq

heap = []
heapq.heappush(heap, 5)
heapq.heappush(heap, 1)
heapq.heappush(heap, 3)

print(heapq.heappop(heap))   # 1  smallest
print(heapq.heappop(heap))   # 3
```

QUESTION

Cost of `heappush`, `heappop`, `heappushpop`, `heapreplace`, `heapify`, `nlargest`, `nsmallest`, `heap[0]`?

heapq functions

Function	Cost	Note
`heappush(heap, x)`	$O(\log n)$	push `x`
`heappop(heap)`	$O(\log n)$	pop the smallest
`heappushpop(heap, x)`	$O(\log n)$	push, then pop smallest (faster than both)
`heapreplace(heap, x)`	$O(\log n)$	pop smallest, then push `x`
`heapify(list)`	$O(n)$	list $\rightarrow$ heap, in place
`nlargest(k, it)`	$O(n \log k)$	the k largest
`nsmallest(k, it)`	$O(n \log k)$	the k smallest
`heap[0]`	$O(1)$	peek the smallest

QUESTION

heapq only gives a min-heap. How do you get a max-heap?

Max-heap trick

- Negate values on the way **in** and again on the way **out**.

```
import heapq

heap = []
heapq.heappush(heap, -5)
heapq.heappush(heap, -1)
heapq.heappush(heap, -3)

print(-heapq.heappop(heap))   # 5  largest
print(-heapq.heappop(heap))   # 3  second largest
```

QUESTION

How do you give items custom priorities in heapq?

Priority queue with tuples

- Push tuples shaped `(priority, value)`; the smallest priority pops first.
- Extra: on equal priorities Python compares the values, which can fail for non-comparable types.

```
import heapq

heap = []
heapq.heappush(heap, (2, "task x"))
heapq.heappush(heap, (1, "task l"))
heapq.heappush(heap, (3, "task c"))

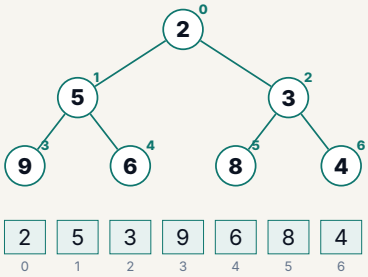
print(heapq.heappop(heap))   # (1, 'task l')
```

QUESTION

How is a binary heap stored in an array? Give the children and parent of index *i*.

Heap as an array

- Left child: $2i + 1$
- Right child: $2i + 2$
- Parent: $(i - 1) // 2$
- `push`: append, then **sift up**.
- `pop`: move the last item to the root, then **sift down**.
- Tree height is $\log n$, so both are $O(\log n)$.



QUESTION

MinHeap (1/4): constructor and the three index helpers.

MinHeap: constructor & index helpers

- The heap lives in a plain list `self.heap`.

```
class MinHeap:
    def __init__(self):
        self.heap = []

    def _parent(self, i):
        return (i - 1) // 2

    def _left(self, i):
        return 2 * i + 1

    def _right(self, i):
        return 2 * i + 2
```

QUESTION

MinHeap (2/4): `push` and `_sift_up`.

MinHeap: push & sift up

- Continues `class MinHeap`.
- Append at the end, then swap upward while the parent is larger.

```
    def push(self, val):
        self.heap.append(val)
        self._sift_up(len(self.heap) - 1)

    def _sift_up(self, i):
        while i > 0 and self.heap[i] < self.heap[self._parent(i)]:
            p = self._parent(i)
            self.heap[i], self.heap[p] = self.heap[p], self.heap[i]
            i = p
```

QUESTION

MinHeap (3/4): `pop`, `peek`, `is_empty`.

MinHeap: pop

- Continues `class MinHeap`.
- Replace the root with the **last** item, shrink the list, then sift down.

```
    def pop(self):
        if not self.heap:
            raise IndexError("Empty Heap")
        top = self.heap[0]
        last = self.heap.pop()
        if self.heap:
            self.heap[0] = last
            self._sift_down(0)
        return top

    def peek(self):
        if not self.heap:
            raise IndexError("Empty Heap")
        return self.heap[0]

    def is_empty(self):
        return len(self.heap) == 0
```

QUESTION

MinHeap (4/4): write `_sift_down`.

MinHeap: sift down

- Continues `class MinHeap`.
- Swap with the **smaller** child until neither child is smaller.

```
    def _sift_down(self, i):
        n = len(self.heap)
        while True:
            smallest = i
            l, r = self._left(i), self._right(i)
            if l < n and self.heap[l] < self.heap[smallest]:
                smallest = l
            if r < n and self.heap[r] < self.heap[smallest]:
                smallest = r
            if smallest == i:
                break
            self.heap[i], self.heap[smallest] = self.heap[smallest], self.heap[i]
            i = smallest
```

09

Binary Trees

Traversals, building a tree from two traversals, and the classic recursive one-liners.

14 cards

QUESTION

Define `TreeNode` and build a small tree by hand.

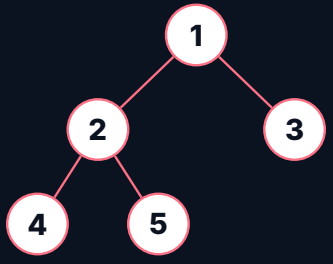
TreeNode basics

```
class TreeNode:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None

tree = TreeNode(1)
tree.left = TreeNode(2)
tree.right = TreeNode(3)
tree.left.left = TreeNode(4)
```

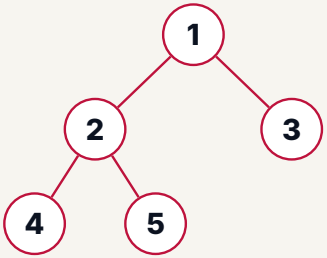
QUESTION

For the tree shown, write the preorder, inorder, postorder and BFS orders.



Traversal orders

- **Preorder** (root, left, right): 1 2 4 5 3
- **Inorder** (left, root, right): 4 2 5 1 3
- **Postorder** (left, right, root): 4 5 2 3 1
- **BFS** (level by level): 1 2 3 4 5
- Memory aid: the root is visited **before / between / after** its children.
- Inorder on a BST gives a **sorted** sequence.



QUESTION

Write the three recursive DFS traversals: preorder, inorder, postorder.

DFS traversals

- The three differ only in where the `print` sits.

```
def preorder_dfs(root: TreeNode) -> None:
    if root:
        print(root.data, end=" ")
        preorder_dfs(root.left)
        preorder_dfs(root.right)

def inorder_dfs(root: TreeNode) -> None:
    if root:
        inorder_dfs(root.left)
        print(root.data, end=" ")
        inorder_dfs(root.right)

def postorder_dfs(root: TreeNode) -> None:
    if root:
        postorder_dfs(root.left)
        postorder_dfs(root.right)
        print(root.data, end=" ")
```

QUESTION

Write BFS (level-order) on a binary tree. Which data structure drives it?

BFS on a tree

- A **queue** (deque): pop from the left, append children on the right.

```
from collections import deque

def bfs(root: TreeNode) -> None:
    if not root:
        return

    queue = deque([root])
    while queue:
        node = queue.popleft()
        print(node.data, end=" ")
        if node.left:
            queue.append(node.left)
        if node.right:
            queue.append(node.right)
```

QUESTION

Write inorder traversal iteratively with an explicit stack.

Iterative inorder

- Go left as far as possible, pushing nodes; pop, visit, then go right.

```
def iterative_traversal(root: TreeNode) -> list:
    result, stack = [], []
    curr = root
    while curr or stack:
        while curr:
            stack.append(curr)
            curr = curr.left
        curr = stack.pop()
        result.append(curr.data)
        curr = curr.right
    return result
```

QUESTION

Given preorder and inorder traversals, how do you rebuild the tree?

Build a tree: the idea

- `preorder[0]` is the **root**.
- Find the root in `inorder`: everything left of it is the left subtree, right of it the right subtree.
- The left subtree's size also tells you how to split `preorder`.
- Recurse on both halves.

QUESTION

Rebuild the tree from preorder + inorder — the naive version. What makes it slow?

Build a tree: naive

- `inorder.index(...)` and list slicing cost $O(n)$ per call, so $O(n^2)$ overall.

```
def build_tree(preorder, inorder):
    if not preorder or not inorder:
        return None

    root_val = preorder[0]
    root = TreeNode(root_val)

    mid = inorder.index(root_val)

    root.left = build_tree(preorder[1:mid + 1], inorder[:mid])
    root.right = build_tree(preorder[mid + 1:], inorder[mid + 1:])

    return root
```

QUESTION

Rebuild the tree from preorder + inorder in $O(n)$. What two tricks make it fast?

Build a tree: optimal

- A hashmap gives each value's inorder index in $O(1)$.
- Pass index bounds, not slices; one moving preorder pointer picks the roots.
- Build the **left** subtree before the right: the pointer only works in that order.

```
def build_tree(preorder, inorder):
    index = {v: i for i, v in enumerate(inorder)}
    pre_idx = 0

    def helper(left, right):
        nonlocal pre_idx
        if left > right:
            return None
        root = TreeNode(preorder[pre_idx])
        pre_idx += 1
        mid = index[root.val]
        root.left = helper(left, mid - 1)
        root.right = helper(mid + 1, right)
        return root

    return helper(0, len(preorder) - 1)
```

QUESTION

Write `height` of a binary tree. What does an empty tree return?

Height of a tree

- Empty tree → `0` (counting nodes on the longest root-to-leaf path).

```
def height(root: TreeNode) -> int:
    if not root:
        return 0

    leftHt = height(root.left)
    rightHt = height(root.right)

    return 1 + max(leftHt, rightHt)
```

QUESTION

Write `numNodes` for a binary tree.

Number of nodes

```
def numNodes(root: TreeNode) -> int:
    if not root:
        return 0

    leftNums = numNodes(root.left)
    rightNums = numNodes(root.right)

    return 1 + leftNums + rightNums
```

QUESTION

Write `isIdentical(p, q)` for two trees. What is the base case?

Are two trees identical?

- If either is None, they match only if **both** are None.

```
def isIdentical(p: TreeNode, q: TreeNode) -> bool:
    if not p or not q:
        return p == q

    return (
        p.data == q.data and
        isIdentical(p.left, q.left) and
        isIdentical(p.right, q.right)
    )
```

QUESTION

Write `isSubtree(root, subRoot)`. How does `isIdentical` help?

Is it a subtree?

- An empty `subRoot` always matches; an empty `root` never does.
- At each node, test `isIdentical`; otherwise search left or right.
- Worst case about $O(m \cdot n)$ for trees of sizes n and m.

```
def isSubtree(root: TreeNode, subRoot: TreeNode) -> bool:
    if not subRoot:
        return True
    if not root:
        return False

    if root.data == subRoot.data and isIdentical(root, subRoot):
        return True

    return (
        isSubtree(root.left, subRoot) or
        isSubtree(root.right, subRoot)
    )
```

QUESTION

Find the diameter of a binary tree. What do you compute at each node, and what do you return upward?

Diameter of a tree

- Diameter = the longest path between **any** two nodes; it need not pass through the root.
- At each node the best path through it is `left + right` (depths) — update the global max.
- Return `1 + max(left, right)` upward. $O(n)$.

```
def diameter(root: TreeNode) -> int:
    result = 0

    def depth(root: TreeNode) -> int:
        nonlocal result
        if not root:
            return 0
        left = depth(root.left)
        right = depth(root.right)
        result = max(result, left + right)
        return 1 + max(left, right)

    depth(root)
    return result
```

QUESTION

Check whether a tree is height-balanced in $O(n)$. What trick avoids recomputing heights?

Is the tree balanced?

- **Balanced:** at every node the left/right heights differ by at most 1.
- Naive: recompute height at each node, $O(n^2)$.
- Better: one bottom-up pass; return `-1` as a sentinel for “unbalanced”.

```python
def is_balanced(root: TreeNode) -> bool:
    def height(node: TreeNode) -> int:
        if not node:
            return 0
        left = height(node.left)
        if left == -1:
            return -1
        right = height(node.right)
        if right == -1:
            return -1
        if abs(left - right) > 1:
            return -1
        return 1 + max(left, right)

    return height(root) != -1
```

10

Binary Search Trees

Ordered trees: insert, search, delete, validate, lowest common ancestor.

8 cards

QUESTION

State the BST property. What does an inorder traversal of a BST produce?

BST property

- For every node: everything in the left subtree $\leq$ node $\leq$ everything in the right subtree.
- Inorder traversal visits the keys in **sorted** order.
- Operations cost $O(h)$: $O(\log n)$ if balanced, $O(n)$ if the tree degenerates into a chain.

QUESTION

Implement a BST (1/4): `TreeNode` and the `BST` constructor.

BST: TreeNode & constructor

```
class TreeNode:
    def __init__(self, val: int):
        self.val = val
        self.left = None
        self.right = None

class BST:
    def __init__(self):
        self.root = None
```

QUESTION

Implement a BST (2/4): `insert`.

BST: insert

- Continues `class BST`.
- Smaller values go left; equal or larger go right (duplicates go right here).

```
    def insert(self, val: int) -> None:
        self.root = self._insert(self.root, val)

    def _insert(self, node, val):
        if node is None:
            return TreeNode(val)
        if val < node.val:
            node.left = self._insert(node.left, val)
        else:
            node.right = self._insert(node.right, val)
        return node
```

QUESTION

Implement a BST (3/4): `search`.

BST: search

- Continues `class BST`.
- Compare, then go left or right: one path down, $O(h)$.

```
    def search(self, val: int) -> TreeNode:
        return self._search(self.root, val)

    def _search(self, node, val):
        if node is None or node.val == val:
            return node
        if val < node.val:
            return self._search(node.left, val)
        return self._search(node.right, val)
```

QUESTION

Deleting a node from a BST: what are the three cases?

BST delete: the cases

- **No left child** → replace the node with its right child.
- **No right child** → replace the node with its left child.
- **Two children** → copy the **inorder successor** (smallest node of the right subtree) into this node, then delete that successor from the right subtree.
- Walk down first, exactly like `search`.

QUESTION

Implement a BST (4/4): write `delete` / `_delete`.

BST: delete

```
    def delete(self, val: int) -> None:
        self.root = self._delete(self.root, val)

    def _delete(self, node, val):
        if node is None:
            return None
        if val < node.val:
            node.left = self._delete(node.left, val)
        elif val > node.val:
            node.right = self._delete(node.right, val)
        else:
            if not node.left:
                return node.right
            if not node.right:
                return node.left
            succ = node.right
            while succ.left:
                succ = succ.left
            node.val = succ.val
            node.right = self._delete(node.right, succ.val)
        return node
```

QUESTION

Validate a BST. Why do you pass `low` and `high` bounds down instead of only comparing a node to its children?

Validate a BST

- Comparing only with children misses violations deeper in a subtree.
- Each node must lie strictly between the bounds inherited from **all** its ancestors.

```
def is_valid_bst(node, low=float('-inf'), high=float('inf')) -> bool:
    if not node:
        return True

    if not (low < node.val < high):
        return False

    return (is_valid_bst(node.left, low, node.val) and
            is_valid_bst(node.right, node.val, high))
```

WATCH OUT This strict check assumes unique keys. If duplicates are allowed (they go right in `insert`), relax one bound to `<=`.

QUESTION

Find the lowest common ancestor of two nodes in a binary tree. What does it mean when both sides return something?

Lowest common ancestor

- Return the node when it equals `p` or `q` (or is `None`).
- If both subtrees return a node, the current node is the LCA.
- Otherwise pass up whichever side found something.

```
def lca(root, p: int, q: int):
    if root is None or root.val == p or root.val == q:
        return root

    left = lca(root.left, p, q)
    right = lca(root.right, p, q)

    if left and right:
        return root

    return left if left else right
```

11

Graphs

Representations, traversals, cycle detection, ordering, components, shortest paths.

18 cards

QUESTION

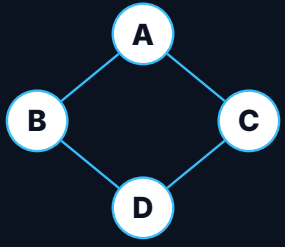
Name the graph types and what each means.

Types of graphs

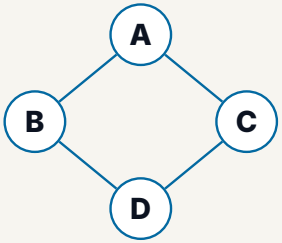
- **Directed / undirected:** edges have a direction / don't.
- **Weighted / unweighted:** edges carry a cost / don't.
- **Cyclic / acyclic:** contains a cycle / doesn't (a DAG is directed and acyclic).
- **Connected:** every node can reach every other node.
- **Sparse / dense:** few edges ($\approx V$) / many edges ($\approx V^2$).

QUESTION

Represent the graph shown as an adjacency list, an adjacency matrix and an edge list.



Three representations



- Nodes A, B, C, D = indices 0, 1, 2, 3.

```
# adjacency list (dict of lists)
graph = {'A': ['B', 'C'], 'B': ['A', 'D'],
         'C': ['A', 'D'], 'D': ['B', 'C']}

# adjacency matrix: 1 = edge i-j
matrix = [[0, 1, 1, 0],
          [1, 0, 0, 1],
          [1, 0, 0, 1],
          [0, 1, 1, 0]]

# edge list (common input format)
edges = [('A', 'B'), ('A', 'C'),
         ('B', 'D'), ('C', 'D')]
```

QUESTION

Adjacency list vs matrix vs edge list: when do you pick each?

Choosing a representation

- **List:** a dict of neighbour lists. Efficient for **sparse** graphs.
- **Matrix:** `matrix[i][j] = 1` if there's an edge. Suits **dense** graphs; V^2 space.
- **Edge list:** a flat list of edges. Rarely used to traverse, but a common way for problems to give input.

QUESTION

How do you store edge weights in an adjacency list?

Weighted adjacency list

- Store `(neighbor, weight)` pairs instead of bare neighbours.

```
graph = {
    'A': [('B', 4), ('C', 1)],
    'B': [('A', 4), ('D', 2)],
    'C': [('A', 1), ('D', 5)],
    'D': [('B', 2), ('C', 5)],
}
```

QUESTION

Build an adjacency list from an edge list, for directed and undirected graphs.

Edge list → adjacency list

- `defaultdict(list)` removes the “is this node in the dict yet?” check.

```
from collections import defaultdict

def build_adj_list(edges: list, directed=False) -> dict:
    graph = defaultdict(list)
    for u, v in edges:
        graph[u].append(v)
        if not directed:
            graph[v].append(u)
    return graph
```

QUESTION

Write recursive DFS on a graph. What stops it from looping forever?

DFS (recursive)

- A `visited` set: mark on entry, only recurse into unvisited neighbours.

```
def dfs(graph, node, visited=None):
    if visited is None:
        visited = set()
    visited.add(node)
    result = [node]
    for neighbor in graph[node]:
        if neighbor not in visited:
            result.extend(dfs(graph, neighbor, visited))
    return result
```

QUESTION

Write iterative DFS with a stack. In what order are neighbours visited?

DFS (iterative)

- Check `visited` when you **pop**, so a node pushed twice is handled once.
- Neighbours are pushed then popped last-in-first-out, so the **last** neighbour is explored first.

```
def dfs_iterative(graph, start):
    visited = set()
    stack = [start]
    result = []
    while stack:
        node = stack.pop()
        if node not in visited:
            visited.add(node)
            result.append(node)
            for neighbor in graph[node]:
                if neighbor not in visited:
                    stack.append(neighbor)
    return result
```

QUESTION

Write BFS on a graph. When do you mark a node as visited — and why then?

BFS

- BFS is always iterative and uses a **queue**.
- Mark visited when you **enqueue**, so no node enters the queue twice.

```
from collections import deque

def bfs(graph, start):
    visited = {start}
    q = deque([start])
    result = []
    while q:
        node = q.popleft()
        result.append(node)
        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                q.append(neighbor)
    return result
```

QUESTION

Detect a cycle in an undirected graph. Why do you pass the `parent` along?

Cycle: undirected graph

- Every edge appears in both directions, so the neighbour you came from is **not** a cycle.
- A visited neighbour that isn't the parent means a cycle.
- Loop over all nodes to cover disconnected components.

```
def detect_cycle_undirected(graph):
    visited = set()

    def dfs(node, parent):
        visited.add(node)
        for neighbor in graph[node]:
            if neighbor not in visited:
                if dfs(neighbor, node):
                    return True
            elif neighbor != parent:
                return True
        return False

    for node in graph:
        if node not in visited:
            if dfs(node, None):
                return True
    return False
```

QUESTION

Detecting a cycle in a directed graph with DFS: what do the three colours mean, and why isn’t a plain visited set enough?

Cycle: directed graph (idea)

- **WHITE** = unvisited, **GRAY** = on the current DFS path, **BLACK** = fully done.
- Reaching a **GRAY** node means a back edge: a cycle.
- Reaching a **BLACK** node is fine — that part is finished — so a plain visited set would give false alarms.

QUESTION

Write cycle detection for a directed graph with the three colours.

Cycle: directed graph (code)

```
def detect_cycle_directed(graph):
    WHITE, GRAY, BLACK = 0, 1, 2
    color = {node: WHITE for node in graph}

    def dfs(node):
        color[node] = GRAY
        for neighbor in graph[node]:
            if color[neighbor] == GRAY:
                return True
            if color[neighbor] == WHITE and dfs(neighbor):
                return True
        color[node] = BLACK
        return False

    for node in graph:
        if color[node] == WHITE:
            if dfs(node):
                return True
    return False
```

QUESTION

Topological sort with DFS: when do you append a node to the stack, and what do you return?

Topological sort (DFS): idea

- Only valid on a **DAG**.
- Append a node **after** all its descendants are done, then return the stack **reversed**.
- Every node must be a key of `graph` (give leaves an empty list), or `graph[neighbor]` raises `KeyError`.

QUESTION

Write topological sort with DFS.

Topological sort (DFS): code

```
def topological_sort(graph):
    visited = set()
    stack = []

    def dfs(node):
        visited.add(node)
        for neighbor in graph[node]:
            if neighbor not in visited:
                dfs(neighbor)
        stack.append(node)   # after all descendants

    for node in graph:
        if node not in visited:
            dfs(node)

    return stack[::-1]
```

QUESTION

Kahn’s algorithm: how does it order a DAG, and how does it reveal a cycle?

Topological sort (Kahn): idea

- Compute every node’s in-degree; start from all nodes with in-degree 0.
- Pop a node, “remove” its outgoing edges, and enqueue neighbours whose in-degree drops to 0.
- If fewer nodes come out than went in, there’s a **cycle**.

QUESTION

Write Kahn’s algorithm (BFS topological sort).

Topological sort (Kahn): code

```python
from collections import deque

def topological_sort_kahn(graph, nodes):
    in_degree = {n: 0 for n in nodes}
    for node in graph:
        for neighbor in graph[node]:
            in_degree[neighbor] += 1

    q = deque([n for n in nodes if in_degree[n] == 0])
    result = []
    while q:
        node = q.popleft()
        result.append(node)
        for neighbor in graph[node]:
            in_degree[neighbor] -= 1
            if in_degree[neighbor] == 0:
                q.append(neighbor)

    if len(result) != len(nodes):
        return None   # cycle: no valid order
    return result
```

QUESTION

Count the connected components of an undirected graph.

Connected components

- Each time DFS starts from an unvisited node, that's a **new component**.

```
def count_components(graph, nodes):
    visited = set()
    count = 0

    def dfs(node):
        visited.add(node)
        for neighbor in graph[node]:
            if neighbor not in visited:
                dfs(neighbor)

    for node in nodes:
        if node not in visited:
            dfs(node)
            count += 1
    return count
```

QUESTION

How does Dijkstra’s algorithm work, what does the `continue` line skip, and what input breaks it?

Dijkstra: idea

- Keep a min-heap of `(distance, node)` and always expand the closest unsettled node.
- `if d > dist[node]: continue` skips **stale** heap entries (a shorter path was already found).
- Only valid for **non-negative** weights. About $O((V + E) \log V)$.

QUESTION

Write Dijkstra’s algorithm with heapq.

Dijkstra’s algorithm

```python
import heapq

def dijkstra(graph, start):
    dist = {start: 0}
    heap = [(0, start)]

    while heap:
        d, node = heapq.heappop(heap)
        if d > dist.get(node, float('inf')):
            continue
        for neighbor, weight in graph[node]:
            new_dist = d + weight
            if new_dist < dist.get(neighbor, float('inf')):
                dist[neighbor] = new_dist
                heapq.heappush(heap, (new_dist, neighbor))
    return dist
```

12

Bit Manipulation

Operators, XOR identities, bit tricks, and the classic XOR / bitmask patterns.

10 cards

QUESTION

For `a = 5` (0101) and `b = 3` (0011), what are `a & b`, `a | b`, `a ^ b`, `~a`, `a << 1`, `a >> 1`?

Basic operators

```
a = 5            # 0101
b = 3            # 0011

a & b            # 1    0001  and
a | b            # 7    0111  or
a ^ b            # 6    0110  xor
~a               # -6   ~a == -(a + 1)
a << 1           # 10   1010  multiply by 2
a >> 1           # 2    0010  divide by 2, floor
```

QUESTION

State the three XOR identities. Which one makes the “cancel out” tricks work?

XOR properties

- XOR-ing the same value twice **cancels** it: `a ^ a ^ b = b` — the engine behind most XOR tricks.

```
a ^ a = 0        # anything with itself is 0
a ^ 0 = a        # with 0 keeps the value
a ^ a ^ b = b    # same value twice cancels out
```

QUESTION

What does ~a really compute in Python, and how do you get an 8-bit flip?

think it through, then flip →

NOT in Python

- Python ints have infinite precision, so `~a` is defined as $-(a+1)$.
- To flip only the low 8 bits, mask the result with `0xFF` (`11111111`).

```
def not_8bit(a):
    return ~a & 0xFF
```

QUESTION

What do `x << n` and `x >> n` compute?

think it through, then flip →

Shifting

- `x << n` is $x \cdot 2^n$.
- `x >> n` is $\lfloor x/2^n \rfloor$.
- So `1 << n` is 2^n.

```
1 << n      # 2^n
x >> n      # floor(x / 2^n)
```

QUESTION

Write one-liners to check, set, clear and toggle the *i*-th bit of n.

i-th bit tricks

- Check: shift down, test the lowest bit.
- Set: OR with a mask. Clear: AND with the inverted mask. Toggle: XOR with the mask.

```
def is_bit_set(n, i):
    return (n >> i) & 1 == 1

def set_bit(n, i):
    return n | (1 << i)

def clear_bit(n, i):
    return n & ~(1 << i)

def toggle_bit(n, i):
    return n ^ (1 << i)
```

QUESTION

Check whether a number is a power of two in $O(1)$. Why does it work?

Power of two

- A power of two has exactly **one** set bit.
- `n - 1` flips that bit and sets everything below it, so `n & (n - 1)` is `0`.
- The `n > 0` guard excludes `0`.

```
def is_power_of_two(n):
    return n > 0 and (n & (n - 1)) == 0
```

QUESTION

Every number appears twice except one. Find it in $O(n)$ time and $O(1)$ space.

XOR: the single number

- XOR everything: pairs cancel, the loner is left.
- This works because the others appear **exactly twice** — not for an arbitrary duplicate.

```
def findSingle(nums: list) -> int:
    ans = nums[0]
    for i in range(1, len(nums)):
        ans ^= nums[i]
    return ans
```

QUESTION

Find the missing number in `0..n` using XOR.

XOR: the missing number

- Start with n, then XOR in every index **and** every value.
- Everything present cancels; the missing number remains.

```
def findMissing(nums: list) -> int:
    result = len(nums)
    for i, num in enumerate(nums):
        result ^= i ^ num
    return result
```

QUESTION

Swap two integers with XOR. What is the Pythonic way?

XOR swap

- In Python prefer `a, b = b, a`.

```
a = a ^ b
b = a ^ b    # (a ^ b) ^ b      = a
a = a ^ b    # (a ^ b) ^ a      = b
```

QUESTION

Generate all subsets of a list with bitmasks. How many masks, and what does each bit mean?

All subsets via bitmask

- 2^n masks from `0` to `(1 << n) - 1`.
- Bit *i* of the mask says whether `nums[i]` is in the subset.
- $O(n \cdot 2^n)$ time.

```python
def allSubsets(nums: list) -> list:
    n = len(nums)
    subsets = []

    for mask in range(1 << n):
        subset = [nums[i] for i in range(n) if mask & (1 << i)]
        subsets.append(subset)

    return subsets
```

STILL EMPTY IN YOUR NOTES

No cards yet for these headings

Prim’s algorithm · Kruskal’s algorithm · Floyd–Warshall · Bellman–Ford · SPFA · Minimum Spanning Tree · Minimum Height Tree

Fill them in and they can become cards in the same format.